

PANDUAN MEMBUAT REST API DENGAN CODEIGNITER 4

Studi Kasus: CRUD Tabel **Mahasiswa** dengan Database PostgreSQL

Mata Kuliah Manajemen Database Client Server (PLMKB502) – D-4 TRPL Politeknik Negeri Medan

Dosen Pengampu: Donny Sanjaya, M.Kom.

Tujuan Pembelajaran

- Memahami konsep REST API dan pemetaan HTTP method ke operasi CRUD.
- Membuat tabel PostgreSQL melalui *migration* dan *seeder* CodeIgniter 4.
- Membangun endpoint CRUD menggunakan *Model*, *ResourceController*, dan *resource route*.
- Menguji API menggunakan Postman dan cURL, termasuk skenario gagal (validasi dan data tidak ditemukan).

Seluruh kode di modul ini sudah diuji pada **CodeIgniter 4.7.4**, **PHP 8.4**, dan **PostgreSQL 16**. Contoh response pada Bagian 9 adalah hasil pengujian yang sebenarnya.

1. Konsep Singkat REST API

REST API adalah antarmuka berbasis HTTP yang memungkinkan **client** (aplikasi web, Flutter, Postman) mengakses data di **server**. Client mengirim *request* (method + URL + body JSON), lalu server memproses ke database dan mengembalikan *response* berupa JSON dan kode status HTTP.

Alur: **Client** → HTTP Request → **Routes** → **Controller** → **Model** → **PostgreSQL** → JSON Response → **Client**

Endpoint yang akan dibuat:

Method	Endpoint	Operasi	Status sukses
GET	/api/mahasiswa	Menampilkan semua data (opsional pencarian ?q=)	200 OK
GET	/api/mahasiswa/{id}	Menampilkan satu data	200 OK
POST	/api/mahasiswa	Menambah data baru	201 Created
PUT / PATCH	/api/mahasiswa/{id}	Mengubah data (PATCH untuk sebagian kolom)	200 OK
DELETE	/api/mahasiswa/{id}	Menghapus data	200 OK

Kode status yang dipakai: **200** sukses, **201** data dibuat, **400** request/validasi salah, **404** data tidak ditemukan, **500** error di server.

2. Persiapan Lingkungan

Kebutuhan	Keterangan
PHP 8.1 atau lebih baru	Aktifkan ekstensi <code>intl</code> , <code>mbstring</code> , <code>pgsql</code> , dan <code>pdo_pgsql</code> di <code>php.ini</code> .
Composer	Untuk membuat project CodeIgniter 4.
PostgreSQL 14+	Beserta pgAdmin atau DBeaver sebagai client database.
Postman / Thunder Client	Untuk menguji API.

Cek PHP dan buat project

```
php -v
php -m | findstr pgsql      # Windows (Linux/Mac: php -m | grep pgsql)

composer create-project codeigniter4/appstarter api-mahasiswa
cd api-mahasiswa
copy env .env              # Linux/Mac: cp env .env
```

Di Laragon/XAMPP, jika pgsql tidak muncul, buka php.ini lalu hapus tanda titik koma di baris ;extension=pgsql dan ;extension=pdo_pgsql, kemudian restart Apache/terminal.

3. Membuat Database dan Konfigurasi .env

Jalankan di pgAdmin / psql

```
CREATE DATABASE db_akademik;
```

Buka file .env di root project, lalu isi seperti berikut (sesuaikan username dan password PostgreSQL Anda):

.env

```
CI_ENVIRONMENT = development

app.baseUrl = 'http://localhost:8080/'

database.default.hostname = localhost
database.default.database = db_akademik
database.default.username = postgres
database.default.password = password_anda
database.default.DBDriver = Postgre
database.default.port     = 5432
database.default.schema   = public
database.default.charset  = utf8
```

Baris database.default.charset = utf8 **wajib** ditambahkan. Tanpa baris ini muncul error invalid value for parameter "client_encoding": "utf8mb4" karena nilai default CI4 adalah charset MySQL.

4. Membuat Tabel dengan Migration dan Seeder

Generate file

```
php spark make:migration CreateMahasiswaTable
php spark make:seeder MahasiswaSeeder
```

```
<?php

namespace App\Database\Migrations;

use CodeIgniter\Database\Migration;

class CreateMahasiswaTable extends Migration
{
    public function up()
    {
        $this->forge->addField([
            'id' => [
                'type'           => 'INT',
                'auto_increment' => true,    // di PostgreSQL menjadi SERIAL
            ],
            'nim' => [
                'type'           => 'VARCHAR',
                'constraint' => 15,
            ],
            'nama' => [
                'type'           => 'VARCHAR',
                'constraint' => 100,
            ],
            'prodi' => [
                'type'           => 'VARCHAR',
                'constraint' => 50,
            ],
            'angkatan' => [
                'type' => 'SMALLINT',
            ],
            'email' => [
                'type'           => 'VARCHAR',
                'constraint' => 100,
                'null'           => true,
            ],
            'created_at' => ['type' => 'TIMESTAMP', 'null' => true],
            'updated_at' => ['type' => 'TIMESTAMP', 'null' => true],
        ]);

        $this->forge->addPrimaryKey('id');
        $this->forge->addUniqueKey('nim');
        $this->forge->createTable('mahasiswa');
    }

    public function down()
    {
        $this->forge->dropTable('mahasiswa', true);
    }
}
```

```

<?php

namespace App\Database\Seeds;

use CodeIgniter\Database\Seeder;

class MahasiswaSeeder extends Seeder
{
    public function run()
    {
        $now = date('Y-m-d H:i:s');
        $data = [
            [
                'nim'      => '2305181001',
                'nama'     => 'Andi Pratama',
                'prodi'    => 'TRPL',
                'angkatan' => 2023,
                'email'    => 'andi@students.polmed.ac.id',
            ],
            [
                'nim'      => '2305181002',
                'nama'     => 'Siti Rahmawati',
                'prodi'    => 'TRPL',
                'angkatan' => 2023,
                'email'    => 'siti@students.polmed.ac.id',
            ],
            [
                'nim'      => '2405181003',
                'nama'     => 'Budi Santoso',
                'prodi'    => 'TRPL',
                'angkatan' => 2024,
                'email'    => 'budi@students.polmed.ac.id',
            ],
        ];

        foreach ($data as &$row) {
            $row['created_at'] = $now;
            $row['updated_at'] = $now;
        }

        $this->db->table('mahasiswa')->insertBatch($data);
    }
}

```

Jalankan migration dan seeder

```

php spark migrate
php spark db:seed MahasiswaSeeder

```

Hasil tabel di PostgreSQL (\d mahasiswa): kolom id bertipe integer dengan default nextval('mahasiswa_id_seq') (SERIAL), PRIMARY KEY pk_mahasiswa, dan UNIQUE constraint pada nim.

5. Membuat Model

Model mengatur tabel, kolom yang boleh diisi (\$allowedFields), timestamp otomatis, dan aturan validasi. Validasi berjalan otomatis setiap kali insert() atau update() dipanggil.

```

<?php

namespace App\Models;

use CodeIgniter\Model;

class MahasiswaModel extends Model
{
    protected $table          = 'mahasiswa';
    protected $primaryKey     = 'id';
    protected $returnType     = 'array';
    protected $allowedFields = ['nim', 'nama', 'prodi', 'angkatan', 'email'];

    // created_at & updated_at diisi otomatis
    protected $useTimestamps = true;
    protected $dateFormat    = 'datetime';

    // Validasi dijalankan otomatis saat insert() dan update()
    protected $validationRules = [
        'id'      => 'permit_empty|is_natural_no_zero',
        'nim'     => 'required|numeric|max_length[15]|is_unique[mahasiswa.nim,id,{id}]',
        'nama'    => 'required|min_length[3]|max_length[100]',
        'prodi'   => 'required|max_length[50]',
        'angkatan' => 'required|integer|greater_than_equal_to[2000]',
        'email'   => 'permit_empty|valid_email|max_length[100]',
    ];

    protected $validationMessages = [
        'nim' => [
            'required' => 'NIM wajib diisi.',
            'numeric'  => 'NIM hanya boleh berisi angka.',
            'is_unique' => 'NIM sudah terdaftar.',
        ],
        'nama' => [
            'required' => 'Nama wajib diisi.',
            'min_length' => 'Nama minimal 3 karakter.',
        ],
        'prodi' => ['required' => 'Prodi wajib diisi.'],
        'angkatan' => ['required' => 'Angkatan wajib diisi.'],
        'email' => ['valid_email' => 'Format email tidak valid.'],
    ];
}

```

- `is_unique[mahasiswa.nim,id,{id}]` memastikan NIM tidak kembar, tetapi mengabaikan baris milik data itu sendiri saat update.
- Saat `update()`, CI4 hanya memvalidasi kolom yang dikirim, sehingga PATCH sebagian kolom tetap lolos.

6. Membuat Controller API

Buat folder `app/Controllers/Api` lalu file `Mahasiswa.php`. Controller mewarisi `ResourceController` yang sudah menyediakan fungsi response seperti `respond()`, `respondCreated()`, `failNotFound()`, dan `failValidationErrors()`.

```

<?php

namespace App\Controllers\Api;

use CodeIgniter\HTTP\Exceptions\HTTPException;
use CodeIgniter\RESTful\ResourceController;

class Mahasiswa extends ResourceController
{
    protected $modelName = 'App\Models\MahasiswaModel';
    protected $format    = 'json';

    // Membaca body JSON; mengembalikan null jika kosong / format JSON salah
    private function getInput(): ?array
    {
        try {
            $input = $this->request->getJSON(true);
        } catch (HTTPException $e) {
            return null;
        }

        return empty($input) ? null : $input;
    }

    // GET /api/mahasiswa (opsional: ?q=kata_kunci)
    public function index()
    {
        $q = $this->request->getGet('q');

        if ($q) {
            $this->model->groupStart()
                ->like('nama', $q, 'both', null, true) // ILIKE (tidak case-sensitive)
                ->orLike('nim', $q)
                ->groupEnd();
        }

        $data = $this->model->orderBy('id', 'ASC')->findAll();

        return $this->respond([
            'status' => 200,
            'message' => 'Daftar mahasiswa',
            'total'   => count($data),
            'data'    => $data,
        ]);
    }

    // GET /api/mahasiswa/{id}
    public function show($id = null)
    {
        $data = $this->model->find($id);

        if (! $data) {
            return $this->failNotFound("Mahasiswa dengan id {$id} tidak ditemukan.");
        }

        return $this->respond([
            'status' => 200,
            'message' => 'Detail mahasiswa',
            'data'    => $data,
        ]);
    }

    // POST /api/mahasiswa
    public function create()
    {
        $input = $this->getInput();

        if ($input === null) {
            return $this->fail('Body request harus berupa JSON dan tidak boleh kosong.', 400);
        }

        if (! $this->model->insert($input)) {

```

```

        return $this->failValidationErrors($this->model->errors());
    }

    return $this->respondCreated([
        'status' => 201,
        'message' => 'Mahasiswa berhasil ditambahkan.',
        'data'    => $this->model->find($this->model->getInsertID()),
    ]);
}

// PUT/PATCH /api/mahasiswa/{id}
public function update($id = null)
{
    if (! $this->model->find($id)) {
        return $this->failNotFound("Mahasiswa dengan id {$id} tidak ditemukan.");
    }

    $input = $this->getInput();

    if ($input === null) {
        return $this->fail('Body request harus berupa JSON dan tidak boleh kosong.', 400);
    }

    $input['id'] = $id; // dibutuhkan rule is_unique[...id,{id}]

    if (! $this->model->update($id, $input)) {
        return $this->failValidationErrors($this->model->errors());
    }

    return $this->respond([
        'status' => 200,
        'message' => 'Mahasiswa berhasil diperbarui.',
        'data'    => $this->model->find($id),
    ]);
}

// DELETE /api/mahasiswa/{id}
public function delete($id = null)
{
    if (! $this->model->find($id)) {
        return $this->failNotFound("Mahasiswa dengan id {$id} tidak ditemukan.");
    }

    $this->model->delete($id);

    return $this->respondDeleted([
        'status' => 200,
        'message' => "Mahasiswa dengan id {$id} berhasil dihapus.",
    ]);
}
}

```

7. Mendaftarkan Routes

Tambahkan kode berikut di bagian bawah `app/Config/Routes.php`. Fungsi `resource()` otomatis membuat route CRUD. Opsi `except` membuang route `new/edit` yang hanya dipakai untuk form HTML.

`app/Config/Routes.php`

```

// REST API Mahasiswa
$routes->group('api', ['namespace' => 'App\Controllers\Api'], static function ($routes) {
    $routes->resource('mahasiswa', ['except' => 'new,edit']);
});

```

Cek route yang terdaftar

```
php spark routes
```

Method	Route	Handler
GET	api/mahasiswa	Api\Mahasiswa::index

Method	Route	Handler
GET	api/mahasiswa/(.*)	Api\Mahasiswa::show/\$1
POST	api/mahasiswa	Api\Mahasiswa::create
PATCH	api/mahasiswa/(.*)	Api\Mahasiswa::update/\$1
PUT	api/mahasiswa/(.*)	Api\Mahasiswa::update/\$1
DELETE	api/mahasiswa/(.*)	Api\Mahasiswa::delete/\$1

8. Menjalankan Server

Terminal

```
php spark serve
```

API dapat diakses di `http://localhost:8080/api/mahasiswa`. Buka alamat tersebut di browser untuk cek cepat endpoint GET.

9. Cara Menguji API

9.1 Pengujian dengan Postman

1. Buat **Collection** baru bernama API Mahasiswa.
2. Pada tab **Variables** collection, tambahkan `base_url = http://localhost:8080/api`.
3. Buat request untuk setiap endpoint, misalnya GET `{{base_url}}/mahasiswa`.
4. Untuk POST, PUT, dan PATCH: pilih tab **Body** → **raw** → **JSON**, lalu tulis data JSON. Header `Content-Type: application/json` akan terisi otomatis.
5. Klik **Send**, lalu periksa **Status** (misalnya 201 Created) dan isi **Body** response.
6. (Opsional) Tambahkan pengujian otomatis di tab **Scripts** → **Post-response**, lalu jalankan semua request lewat **Run Collection**.

Contoh test script Postman (request POST)

```
pm.test("Status 201 Created", () => {
  pm.response.to.have.status(201);
});

pm.test("NIM sesuai input", () => {
  const res = pm.response.json();
  pm.expect(res.data.nim).to.eql("2405181004");
});
```

9.2 Pengujian dengan cURL

Jalankan di Git Bash, terminal Linux, atau Mac. Di Windows CMD/PowerShell lebih mudah memakai Postman karena aturan tanda kutipnya berbeda.

A. GET semua data – status yang diharapkan: 200

Request

```
curl http://localhost:8080/api/mahasiswa
```

Response

```
{
  "status": 200,
  "message": "Daftar mahasiswa",
  "total": 3,
  "data": [
    {
      "id": "1",
      "nim": "2305181001",
      "nama": "Andi Pratama",
      "prodi": "TRPL",
      "angkatan": "2023",
      "email": "andi@students.polmed.ac.id",
      "created_at": "2026-09-23 14:35:37",
      "updated_at": "2026-09-23 14:35:37"
    },
    ...
  ]
}
```

B. GET dengan pencarian – status yang diharapkan: 200

Request

```
curl "http://localhost:8080/api/mahasiswa?q=siti"
```

Response

```
{
  "status": 200,
  "message": "Daftar mahasiswa",
  "total": 1,
  "data": [ { "id": "2", "nim": "2305181002", "nama": "Siti Rahmawati", ... } ]
}
```

C. GET detail berdasarkan id – status yang diharapkan: 200

Request

```
curl http://localhost:8080/api/mahasiswa/2
```

Response

```
{
  "status": 200,
  "message": "Detail mahasiswa",
  "data": {
    "id": "2",
    "nim": "2305181002",
    "nama": "Siti Rahmawati",
    ...
  }
}
```

D. POST tambah data – status yang diharapkan: 201

Request

```
curl -X POST http://localhost:8080/api/mahasiswa \
-H "Content-Type: application/json" \
-d '{"nim":"2405181004","nama":"Rina Marpaung","prodi":"TRPL",
    "angkatan":2024,"email":"rina@students.polmed.ac.id"}'
```

Response

```
{
  "status": 201,
  "message": "Mahasiswa berhasil ditambahkan.",
  "data": {
    "id": "4",
    "nim": "2405181004",
    "nama": "Rina Marpaung",
    "prodi": "TRPL",
    "angkatan": "2024",
    "email": "rina@students.polmed.ac.id",
    "created_at": "2026-09-23 14:35:46",
    "updated_at": "2026-09-23 14:35:46"
  }
}
```

E. POST data tidak valid (uji validasi) – status yang diharapkan: 400

Request

```
curl -X POST http://localhost:8080/api/mahasiswa \
-H "Content-Type: application/json" \
-d '{"nim":"2305181001","nama":"Ab","email":"salah-format"}'
```

Response

```
{
  "status": 400,
  "error": 400,
  "messages": {
    "nim": "NIM sudah terdaftar.",
    "nama": "Nama minimal 3 karakter.",
    "prodi": "Prodi wajib diisi.",
    "angkatan": "Angkatan wajib diisi.",
    "email": "Format email tidak valid."
  }
}
```

F. PUT ubah seluruh data – status yang diharapkan: 200

Request

```
curl -X PUT http://localhost:8080/api/mahasiswa/4 \
-H "Content-Type: application/json" \
-d '{"nim":"2405181004","nama":"Rina Marpaung, S.Tr.Kom","prodi":"TRPL",
    "angkatan":2024,"email":"rina.m@students.polmed.ac.id"}'
```

Response

```
{
  "status": 200,
  "message": "Mahasiswa berhasil diperbarui.",
  "data": { "id": "4", "nama": "Rina Marpaung, S.Tr.Kom", ... }
}
```

G. PATCH ubah sebagian kolom – status yang diharapkan: 200

Request

```
curl -X PATCH http://localhost:8080/api/mahasiswa/4 \
-H "Content-Type: application/json" -d '{"prodi":"MI"}'
```

Response

```
{
  "status": 200,
  "message": "Mahasiswa berhasil diperbarui.",
  "data": { "id": "4", "prodi": "MI", ... }
}
```

H. PUT dengan NIM milik mahasiswa lain – status yang diharapkan: 400

Request

```
curl -X PUT http://localhost:8080/api/mahasiswa/4 \
-H "Content-Type: application/json" -d '{"nim":"2305181002"}'
```

Response

```
{
  "status": 400,
  "error": 400,
  "messages": { "nim": "NIM sudah terdaftar." }
}
```

I. DELETE hapus data – status yang diharapkan: 200

Request

```
curl -X DELETE http://localhost:8080/api/mahasiswa/4
```

Response

```
{
  "status": 200,
  "message": "Mahasiswa dengan id 4 berhasil dihapus."
}
```

J. GET data yang sudah dihapus / tidak ada – status yang diharapkan: 404

Request

```
curl http://localhost:8080/api/mahasiswa/4
```

Response

```
{
  "status": 404,
  "error": 404,
  "messages": { "error": "Mahasiswa dengan id 4 tidak ditemukan." }
}
```

K. POST dengan body kosong atau JSON rusak – status yang diharapkan: 400

Request

```
curl -X POST http://localhost:8080/api/mahasiswa \
-H "Content-Type: application/json" -d '{"nim:'
```

Response

```
{
  "status": 400,
  "error": 400,
  "messages": { "error": "Body request harus berupa JSON dan tidak boleh kosong." }
}
```

Nilai id dan angkatan dikembalikan sebagai string (misalnya "4") karena driver PostgreSQL PHP mengembalikan semua kolom sebagai teks. Jika perlu angka, gunakan \$casts pada Model, misalnya ['id' => 'int', 'angkatan' => 'int'].

9.3 Lembar Checklist Pengujian

Isi kolom **Status Aktual** dan **Hasil** (Lulus/Gagal) saat melakukan pengujian, lalu lampirkan screenshot Postman sebagai bukti.

No	Skenario	Method	Status Diharapkan	Status Aktual	Hasil
1	GET semua data	GET	200		
2	GET dengan pencarian	GET	200		
3	GET detail berdasarkan id	GET	200		
4	POST tambah data	POST	201		
5	POST data tidak valid (uji validasi)	POST	400		
6	PUT ubah seluruh data	PUT	200		
7	PATCH ubah sebagian kolom	PATCH	200		
8	PUT dengan NIM milik mahasiswa lain	PUT	400		
9	DELETE hapus data	DELETE	200		
10	GET data yang sudah dihapus / tidak ada	GET	404		
11	POST dengan body kosong atau JSON rusak	POST	400		

10. Troubleshooting

Masalah	Penyebab	Solusi
client_encoding: "utf8mb4"	Charset default C14 untuk MySQL.	Tambahkan database.default.charset = utf8 di .env.
Unable to connect to the database	Service PostgreSQL mati, atau username/password/port salah.	Jalankan service PostgreSQL, lalu cek kembali isi .env.
Call to undefined function pg_connect()	Ekstensi pgsql belum aktif.	Aktifkan extension=pgsql di php.ini, lalu restart.
404 pada /api/mahasiswa	Route belum terdaftar atau namespace salah.	Cek dengan php spark routes, lalu cek namespace controller.
Response berupa halaman HTML error	Terjadi error PHP (mode development).	Baca pesan error, lalu cek writable/logs.
Data POST selalu "wajib diisi"	Body dikirim sebagai form-data, bukan JSON.	Di Postman pilih Body → raw → JSON.
relation "mahasiswa" does not exist	Migration belum dijalankan.	Jalankan php spark migrate.

11. Latihan Mandiri

1. Tambahkan kolom no_hp (VARCHAR 15) melalui migration baru, lalu perbarui \$allowedFields dan aturan validasinya.

2. Tambahkan pagination pada endpoint GET menggunakan `$this->model->paginate(10)` beserta informasi pager.
3. Terapkan *soft delete* (`$useSoftDeletes = true` dan kolom `deleted_at`), lalu uji ulang skenario DELETE.
4. Buat endpoint GET `/api/mahasiswa/angkatan/{tahun}` untuk menampilkan data berdasarkan angkatan.
5. Dokumentasikan semua endpoint dalam Postman Collection, lalu ekspor file `.json`-nya sebagai bukti pengujian.